

[VirtualBox Ubuntu 서버/클라이언트 네트워크 실습]

우분투 VM = 서버

내 PC = 클라이언트

VirtualBox 네트워크 = NAT + 호스트 전용 어댑터

<전체 목표>

내 PC = 클라이언트

|
| ping / SSH / 웹 접속



Ubuntu VM = 서버

VirtualBox 네트워크는 다음처럼 구성합니다.

어댑터 1: NAT

- Ubuntu VM이 인터넷에 접속하기 위한 용도

어댑터 2: 호스트 전용 어댑터

- 내 PC와 Ubuntu VM이 서로 통신하기 위한 용도

0단계. 사전 준비

0-1. VirtualBox 네트워크 설정

Ubuntu VM을 완전히 종료한 뒤 VirtualBox에서 설정합니다.

Ubuntu VM 선택

→ 설정

→ 네트워크

어댑터 1

어댑터 사용: 체크

연결 방식: NAT

어댑터 2

어댑터 사용: 체크

연결 방식: 호스트 전용 어댑터

이름: VirtualBox Host-Only Ethernet Adapter

설정 후 Ubuntu VM을 실행합니다.

0-2. Ubuntu에서 IP 확인

Ubuntu 터미널에서 실행합니다.

ip addr

또는 간단히:

hostname -I

보통 다음처럼 IP가 2개 이상 나옵니다.

10.0.2.15 ← NAT IP

192.168.56.101 ← 호스트 전용 어댑터 IP

이번 실습에서는 **192.168.56.x 형태의 IP**를 사용합니다.

예시로 아래 IP를 사용하겠습니다.

Ubuntu 서버 IP: 192.168.56.101

본인 환경에서는 실제 IP로 바꿔서 사용하면 됩니다.

1단계. ping 테스트

목표

내 PC에서 Ubuntu VM까지 네트워크 연결이 되는지 확인합니다.

1-1. Ubuntu VM에서 IP 확인

Ubuntu에서 실행합니다.

```
hostname -I
```

예시 결과:

```
10.0.2.15 192.168.56.101
```

여기서 사용할 IP는 보통:

```
192.168.56.101
```

1-2. 내 PC에서 ping 테스트

Windows PowerShell

```
ping 192.168.56.101
```

macOS / Linux 터미널

```
ping 192.168.56.101
```

정상이라면 다음과 비슷하게 나옵니다.

```
Reply from 192.168.56.101: bytes=32 time<1ms TTL=64
```

또는:

```
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.5 ms
```

1-3. 확인 할 내용

Q1. 내 PC는 클라이언트인가 서버인가?

A1. 클라이언트

Q2. Ubuntu VM은 클라이언트인가 서버인가?

A2. 서버

Q3. ping은 무엇을 확인하는 명령인가?

A3. 상대방 컴퓨터까지 네트워크 연결이 되는지 확인하는 명령

1-4. 오류 상황

ping이 안 될 때 확인할 것

Ubuntu에서 IP를 다시 확인합니다.

```
hostname -I
```

VirtualBox 설정에서 어댑터 2가 켜져 있는지 확인합니다.

설정 → 네트워크 → 어댑터 2

연결 방식: 호스트 전용 어댑터

Ubuntu 방화벽 상태도 확인합니다.

```
sudo ufw status
```

처음에는 방화벽이 꺼져 있어도 괜찮습니다.

2단계. SSH 접속

목표

내 PC에서 Ubuntu VM에 원격 접속합니다.

내 PC → SSH → Ubuntu VM

SSH는 서버를 원격으로 관리할 때 가장 기본적으로 사용하는 접속 방식입니다.

2-1. Ubuntu에 SSH 서버 설치

Ubuntu VM에서 실행합니다.

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

SSH 서버 실행 상태 확인:

```
sudo systemctl status ssh
```

정상이라면 다음과 비슷하게 보입니다.

active (running)

2-2. SSH 서버 시작

혹시 실행 중이 아니라면 다음을 실행합니다.

```
sudo systemctl start ssh
```

부팅할 때 자동 실행되도록 설정합니다.

```
sudo systemctl enable ssh
```

2-3. Ubuntu 사용자 이름 확인

Ubuntu에서 현재 사용자 이름을 확인합니다.

```
whoami
```

예시: poly

그러면 SSH 접속 형식은 다음과 같습니다.

ssh 사용자명@Ubuntu_IP

예: ssh poly@192.168.56.101

2-4. 내 PC에서 SSH 접속

Windows PowerShell

```
ssh poly@192.168.56.101
```

macOS / Linux 터미널

```
ssh poly@192.168.56.101
```

처음 접속하면 다음과 같은 메시지가 나올 수 있습니다.

Are you sure you want to continue connecting?

여기서 입력합니다. : yes

그 다음 Ubuntu 비밀번호를 입력합니다.

2-5. 접속 성공 확인

SSH 접속에 성공하면 내 PC 터미널에서 Ubuntu 명령어를 실행할 수 있습니다.

```
hostname
```

```
pwd
```

```
ls
```

접속 종료: exit

2-6. 핵심 개념 정리
SSH 서버: Ubuntu VM
SSH 클라이언트: 내 PC
SSH 기본 포트: 22번

3단계. nginx 웹 서버 접속

목표

Ubuntu VM에 웹 서버를 설치하고, 내 PC 브라우저에서 접속합니다.
내 PC 브라우저 → HTTP → Ubuntu nginx 서버

3-1. nginx 설치

Ubuntu VM에서 실행합니다.

```
sudo apt update
```

```
sudo apt install nginx -y
```

nginx 실행 상태 확인:

```
sudo systemctl status nginx
```

정상 상태:

active (running)

3-2. nginx 시작 및 자동 실행 설정

```
sudo systemctl start nginx
```

```
sudo systemctl enable nginx
```

3-3. Ubuntu 안에서 먼저 테스트

Ubuntu VM에서 실행합니다.

```
curl http://localhost
```

HTML 내용이 출력되면 nginx가 정상 동작 중입니다.

또는 Ubuntu 브라우저에서 접속합니다.

http://localhost

3-4. 내 PC에서 웹 접속

내 PC 브라우저 주소창에 입력합니다.

http://192.168.56.101

nginx 기본 페이지가 나오면 성공입니다.

3-5. 웹 서버 포트 확인

HTTP 기본 포트는 80번입니다.

Ubuntu에서 확인합니다.

```
sudo ss -tulnp | grep nginx
```

또는:

```
sudo ss -tulnp | grep :80
```

예시:

```
LISTEN 0 511 0.0.0.0:80
```

3-6. 핵심 개념 정리

웹 서버: Ubuntu VM

웹 클라이언트: 내 PC 브라우저

HTTP 기본 포트: 80번

HTTPS 기본 포트: 443번

4단계. 방화벽 ufw 설정

목표

Ubuntu의 방화벽을 켜고, 필요한 포트만 허용하는 연습을 합니다.

방화벽은 서버 보안에서 매우 중요합니다.

허용한 포트만 접속 가능

허용하지 않은 포트는 차단

4-1. ufw 상태 확인

Ubuntu에서 실행합니다.

```
sudo ufw status
```

처음에는 보통 다음처럼 나옵니다.

```
Status: inactive
```

4-2. SSH 먼저 허용

방화벽을 켜기 전에 반드시 SSH를 먼저 허용합니다.

```
sudo ufw allow ssh
```

또는 포트 번호로 허용할 수도 있습니다.

```
sudo ufw allow 22/tcp
```

중요합니다.

SSH를 허용하지 않고 방화벽을 켜면

내 PC에서 SSH 접속이 막힐 수 있습니다.

4-3. HTTP 허용

nginx 웹 접속을 위해 80번 포트를 허용합니다.

```
sudo ufw allow 80/tcp
```

또는 서비스 이름으로:

```
sudo ufw allow 'Nginx HTTP'
```

4-4. 방화벽 켜기

```
sudo ufw enable
```

확인 질문이 나오면: y

4-5. 방화벽 상태 확인

```
sudo ufw status
```

예시:

```
Status: active
```

To	Action	From
--	-----	-----
22/tcp	ALLOW	Anywhere
80/tcp	ALLOW	Anywhere

4-6. 접속 테스트

내 PC에서 SSH 접속:

```
ssh poly@192.168.56.101
```

내 PC 브라우저에서 웹 접속:

```
http://192.168.56.101
```

둘 다 정상 접속되면 성공입니다.

5단계. 포트 열기/닫기 실습

목표

서버에서 포트를 허용하거나 차단하면 클라이언트 접속 결과가 어떻게 달라지는지 확인합니다.

5-1. 현재 열린 포트 확인

Ubuntu에서 실행합니다.

```
sudo ss -tulnp
```

주요 확인 대상:

22번 포트: SSH

80번 포트: HTTP

간단히 확인하려면:

```
sudo ss -tulnp | grep -E ':22|:80'
```

5-2. 80번 포트 차단

nginx는 실행 중이지만 방화벽에서 80번 포트를 막아봅니다.

```
sudo ufw deny 80/tcp
```

상태 확인:

```
sudo ufw status
```

5-3. 내 PC에서 웹 접속 테스트

브라우저에서 다시 접속합니다.

<http://192.168.56.101>

정상이라면 접속이 되지 않아야 합니다.

즉:

nginx 서버는 켜져 있음

하지만 방화벽이 80번 포트를 차단함

그래서 클라이언트가 접속할 수 없음

5-4. 80번 포트 다시 열기

먼저 deny 규칙을 삭제합니다.

```
sudo ufw status numbered
```

예시:

```
[1] 22/tcp ALLOW IN Anywhere
```

```
[2] 80/tcp DENY IN Anywhere
```

2번 규칙을 삭제하려면:

```
sudo ufw delete 2
```

그 다음 80번 포트를 다시 허용합니다.

```
sudo ufw allow 80/tcp
```

확인:

```
sudo ufw status
```

내 PC 브라우저에서 다시 접속:

<http://192.168.56.101>

5-5. nginx 서비스 중지 실습

이번에는 방화벽이 아니라 웹 서버 자체를 꺼봅니다.

sudo systemctl stop nginx

내 PC에서 접속:

http://192.168.56.101

접속이 되지 않아야 합니다.

다시 nginx 시작:

sudo systemctl start nginx

5-6. 차이점 정리

상황	nginx 상태	방화벽 80번	접속 결과
정상	실행 중	허용	접속 가능
방화벽 차단	실행 중	차단	접속 불가
서버 중지	중지	허용	접속 불가

핵심 : 서비스가 실행 중이어야 하고, 방화벽에서도 해당 포트가 열려 있어야 클라이언트가 접속할 수 있다.

6단계. VM 하나 더 복제해서 서버-클라이언트 분리

목표

이번에는 내 PC를 클라이언트로 쓰는 것이 아니라, Ubuntu VM을 하나 더 만들어서 다음 구조로 실습합니다.

Ubuntu Server VM Ubuntu Client VM
192.168.56.101 ← → 192.168.56.102

이 구성이 실제 서버/클라이언트 실습에 더 가깝습니다.

6-1. 기존 Ubuntu VM 종료

복제하기 전에 Ubuntu VM을 완전히 종료합니다.

Ubuntu에서: sudo shutdown now

또는 VirtualBox에서 정상 종료합니다.

6-2. VirtualBox에서 VM 복제

VirtualBox 관리자 화면에서:

Ubuntu VM 우클릭 → 복제

권장 설정:

이름: Ubuntu-Client

MAC 주소 정책: 모든 네트워크 어댑터의 새 MAC 주소 생성

복제 방식: 완전한 복제

기존 VM 이름은 다음처럼 바꾸면 좋습니다.

Ubuntu-Server

Ubuntu-Client

6-3. 두 VM의 네트워크 설정

두 VM 모두 같은 방식으로 설정합니다.

Ubuntu-Server

어댑터 1: NAT

어댑터 2: 호스트 전용 어댑터

Ubuntu-Client

어댑터 1: NAT

어댑터 2: 호스트 전용 어댑터

6-4. 두 VM 실행 후 IP 확인

각 VM에서 실행합니다.

```
hostname -I
```

예시:

Ubuntu-Server

10.0.2.15 192.168.56.101

Ubuntu-Client

10.0.2.16 192.168.56.102

이번 실습에서는 다음처럼 가정합니다.

서버 VM IP: 192.168.56.101

클라이언트 VM IP: 192.168.56.102

6-5. 서버 VM에서 SSH와 nginx 준비

Ubuntu-Server에서 실행합니다.

```
sudo apt update
```

```
sudo apt install openssh-server nginx -y
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

```
sudo systemctl enable nginx
```

```
sudo systemctl start nginx
```

방화벽 설정:

```
sudo ufw allow ssh
```

```
sudo ufw allow 80/tcp
```

```
sudo ufw enable
```

```
sudo ufw status
```

6-6. 클라이언트 VM에서 서버로 ping

Ubuntu-Client에서 실행합니다.

```
ping 192.168.56.101
```

정상이라면 응답이 옵니다.

중지하려면:

Ctrl + C

6-7. 클라이언트 VM에서 서버로 SSH 접속

Ubuntu-Client에서 실행합니다.

```
ssh 서버사용자명@192.168.56.101
```

예:

```
ssh poly@192.168.56.101
```

성공하면 Client VM에서 Server VM으로 원격 접속한 것입니다.

나가기:

```
exit
```

6-8. 클라이언트 VM에서 서버 웹 접속

Ubuntu-Client에서 실행합니다.

```
curl http://192.168.56.101
```

HTML이 출력되면 성공입니다.

GUI 환경이면 브라우저에서 접속해도 됩니다.

```
http://192.168.56.101
```


<<전체 실습 요약>>

1. VirtualBox 네트워크 설정
 - NAT
 - 호스트 전용 어댑터
2. Ubuntu 서버 IP 확인
 - hostname -I
 - 192.168.56.x 확인
3. 내 PC에서 ping
 - ping 서버IP
4. SSH 서버 설치
 - openssh-server
 - ssh 사용자명@서버IP
5. nginx 웹 서버 설치
 - nginx
 - http://서버IP
6. ufw 방화벽 설정
 - 22번 허용
 - 80번 허용
7. 포트 열기/닫기
 - 80번 차단
 - 웹 접속 실패 확인
 - 80번 다시 허용
8. VM 복제
 - Ubuntu-Server
 - Ubuntu-Client
9. VM끼리 통신
 - Client VM에서 Server VM으로 ping
 - SSH
 - HTTP 접속